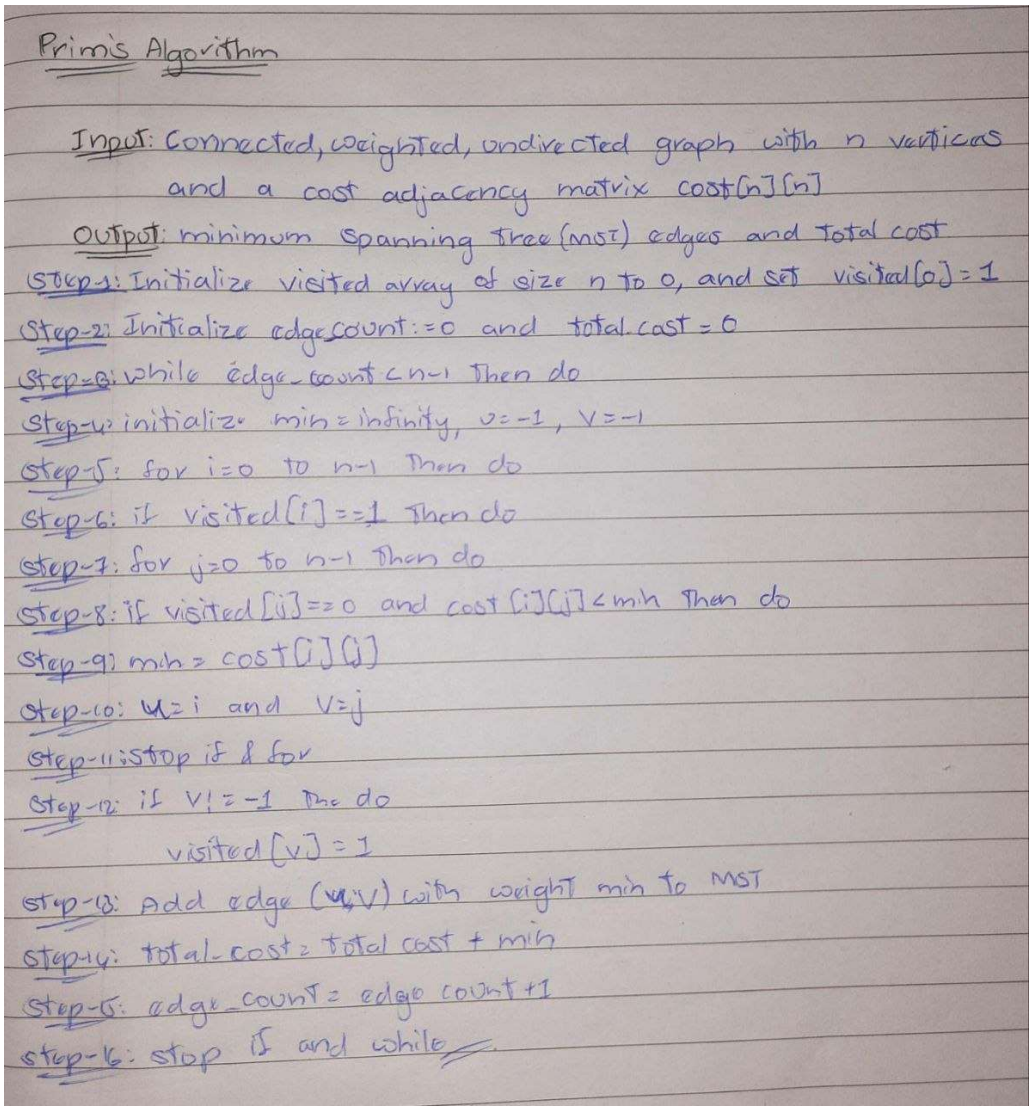
PRACTICAL-09

AIM: Implement prim's algorithm.

ALGORITHM:



PROGRAM:

```
#include <iostream>
#include <climits>
using namespace std;
```

```
int main() {
    int n;
    cout << "Enter number of vertices: ";
    cin >> n;
    int cost[20][20];
    cout << "Enter cost adjacency matrix:\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> cost[i][j];
        }
    }
    int visited[20] = {0};
    visited[0] = 1;
    int edge_count = 0;
    int total_cost = 0;
    cout << "\nEdges in Minimum Spanning Tree:\n";
    while (edge_count < n - 1) {
        int min = INT_MAX;
        int u = -1, v = -1;

        for (int i = 0; i < n; i++) {
            if (visited[i] == 1) {
                for (int j = 0; j < n; j++) {
                    if (visited[j] == 0 &&
                        cost[i][j] != 0 &&
                        cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
```

```
            }
          }
        }
    }
    if (v != -1) {
        visited[v] = 1;
        cout << u << " - " << v
             << " : " << min << endl;
        total_cost += min;
        edge_count++;
    } else {
        cout << "Graph is not connected.\n";
        return 0;
    }
  }
  cout << "Total cost of MST = " << total_cost << endl;
  return 0;
}
```

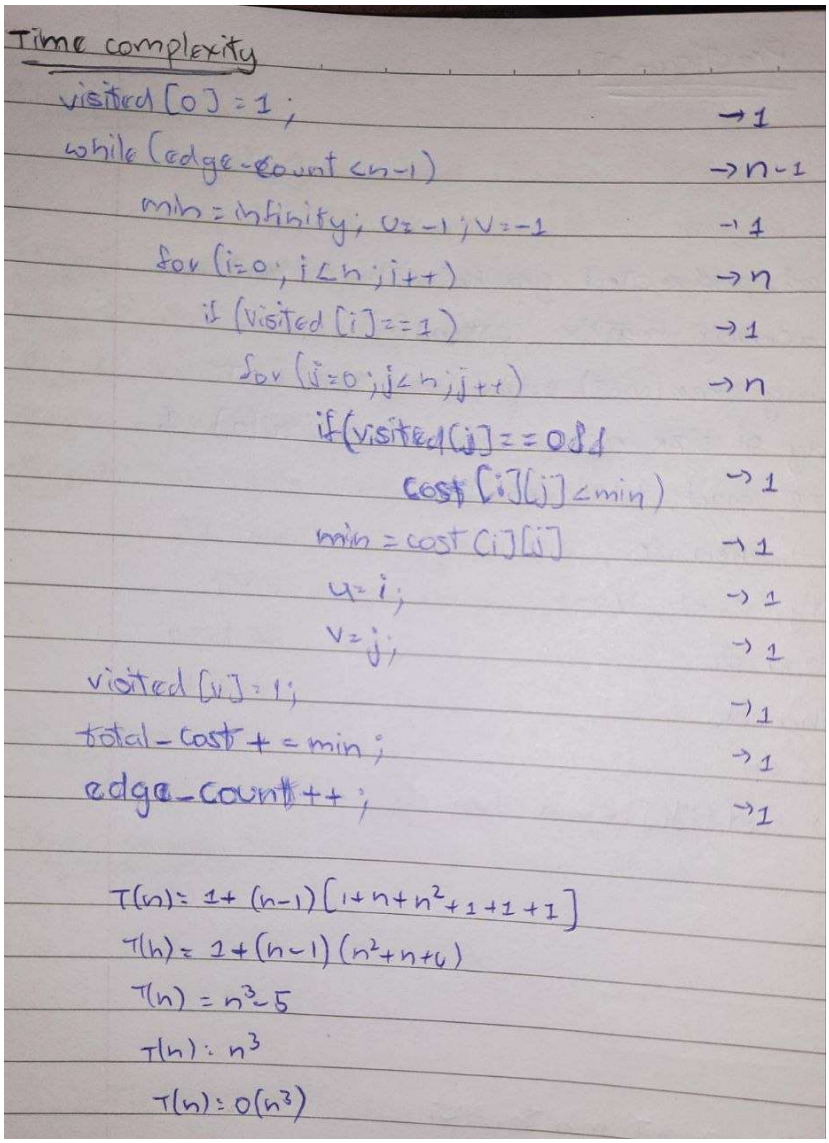
OUTPUT:

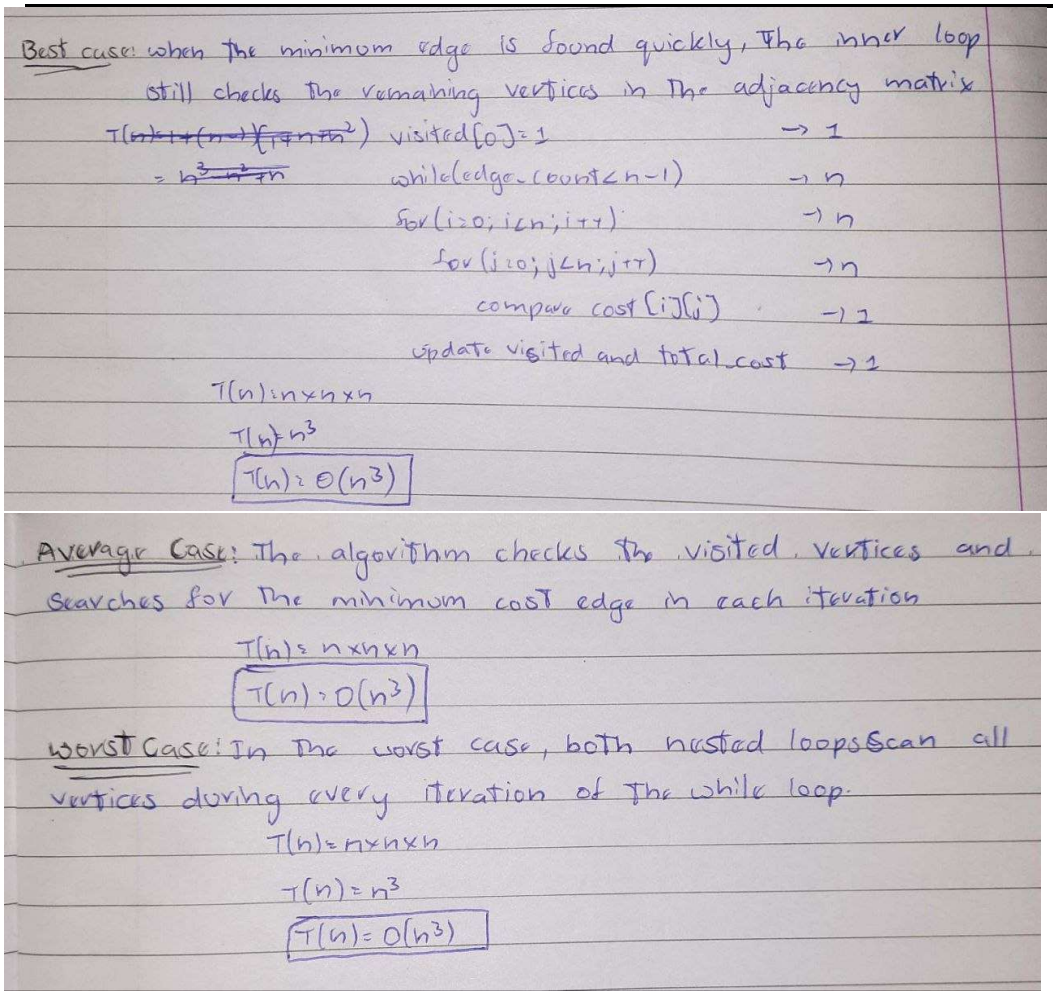
```
Output

Enter number of vertices: 4
Enter cost adjacency matrix:
0 2 0 6
2 0 3 8
0 3 0 1
6 8 1 0

Edges in Minimum Spanning Tree:
0 - 1 : 2
1 - 2 : 3
2 - 3 : 1
Total cost of MST = 6
```

TIME COMPLEXITY:





CONCLUSION: Prim's Algorithm is used to find the Minimum Spanning Tree of a connected, weighted graph. It selects the minimum-cost edge at each step without forming a cycle. It connects all vertices with the minimum total cost. Thus, Prim's Algorithm helps to find an efficient way to connect all vertices in a graph.